

# Performance and Energy Comparison of Four Controllers for a Self-Balancing Cart–Pole

Eirik Bjørnson Jahr  
Department of Informatics  
University of Oslo, Norway  
eiribja@ifi.uio.no

Selen Goktay  
Department of Informatics  
University of Oslo, Norway  
seleng@ifi.uio.no

Elisabeth Chi Fang Jin  
Department of Informatics  
University of Oslo, Norway  
ecjin@ifi.uio.no

**Abstract**—This paper evaluates the four control strategies LQR, MPC, Fuzzy-PD and  $H_\infty$  for stabilizing an inverted pendulum on a cart in a PyBullet simulation. We describe how each controller is designed, how they operate, and use this to interpret the observed performance differences. Performance is evaluated using settling time and control effort. The results highlight clear trade-offs between disturbance rejection, settling speed, and control effort across the four designs. Additionally, we demonstrate MPC swing-up and stabilization.

**Index Terms**—control, energy, MPC, LQR,  $H_\infty$ , Fuzzy-PD

## I. INTRODUCTION

Balancing a robot involves the interaction between sensors, actuators, system inputs/outputs and a well-programmed control algorithm [1]. This is important for robotic systems, ranging from humanoid robots like Memo by Sunday Robotics to simple Arduino-based builds [2] [3]. The self-balancing robot is familiar to hobbyists, students, and university professors alike partly because its dynamics are convenient to model. It is commonly approximated as an inverted pendulum on a cart and, for small angular deviations, treated as a linear system [4].

Many online prototypes use Proportional–Integral–Derivative (PID) control as a practical solution, often excluding alternatives that address optimization objectives or robustness. At the same time, some research has explored several non-PID controllers for balancing tasks. Studt et al. proposed a hybrid optimization-based approach for a ballbot, where the yz-plane is controlled using Linear Quadratic Regulator (LQR) and the xz-plane is controlled using Model Predictive Control (MPC) [5]. Fikri et al. studied a two-wheeled self-balancing robot and compared LQR against an  $H_\infty$  controller using external disturbance inputs [6]. Anisimov et al. compared an LQR- and Fuzzy-PD-controller for a two-wheeled self-balancing robot. [7] While these works highlight useful trade-offs such as differences in angular deviation and disturbance response, the results are reported on different systems. This makes direct comparisons difficult.

The main contribution of this project is a compact comparison of multiple non-PID controllers for a simplified balancing task. Using the libraries Python Control and Scipy, we provide a straightforward implementation for controllers such as LQR, MPC and  $H_\infty$ . Additionally, by using the Scikit-Fuzzy library, we explore a Mamdani-type Fuzzy controller as a non-linear

alternative. Although the Fuzzy controller uses PD-based principles, it expands its functionality with a rule-based approach. The implementations have been refined and improved by using ChatGPT. The goal is to assess how these controllers differ in stabilization, performance and control effort, and to clarify the trade-offs associated with each method.

The remainder of this report is organized as follows. Problem Statement presents the system model, the assumptions used for simulation, the energy metric definition and the simulation conditions. Controller Design outlines the controller implementation details for LQR, MPC,  $H_\infty$ , and the Fuzzy-PD controller. Result and Discussion displays and evaluates the system responses which consist of the angular displacement, the control energy and the settling time. The Proof provides additional background for the theoretical material. The Conclusion summarizes the project.

## II. PROBLEM STATEMENT

We consider a self-balancing robot modeled as an inverted pendulum on a cart. The cart is constrained to one-dimensional horizontal motion and actuated by a horizontal force  $F(t)$ . The measured outputs are the cart position  $p(t)$  and pendulum angle  $\theta(t)$ . The dynamic equations of the robot were derived based on [4]; however, the parameter values used in this work differ from the source. In Figure 1, the model of the robot in the Python simulation environment Pybullet is shown. In Table I, the variables of the dynamic model are presented with their default values.

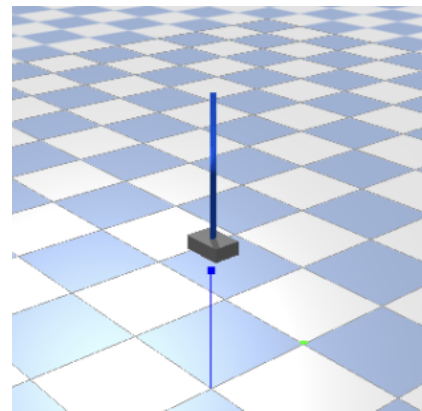


Fig. 1. Inverted Pendulum on a Cart in the Pybullet Simulation Environment

TABLE I  
VARIABLES USED IN THE INVERTED PENDULUM ON A CART MODEL.

| Name of variable               | Symbol      | Value     | Unit              |
|--------------------------------|-------------|-----------|-------------------|
| Mass of cart                   | $M$         | 0.5       | kg                |
| Mass of pole                   | $m$         | 0.5       | kg                |
| Friction coefficient of cart   | $b$         | 0 or 0.15 | N/(m·s)           |
| Length to pole center of mass  | $L$         | 0.5       | m                 |
| Mass moment of inertia         | $I$         | 0.042     | kg·m <sup>2</sup> |
| External force applied to cart | $F(t)$      | 5         | N                 |
| Cart position                  | $x(t)$      | 0         | m                 |
| Pole angle                     | $\theta(t)$ | 0         | rad               |
| Gravity                        | $g$         | 9.81      | m/s <sup>2</sup>  |

The model should be evaluated as a free-body diagram to represent all the forces present. The derivation of the vertical and horizontal normal reaction forces and their contribution to the equations of motion are shown in [4] [8].

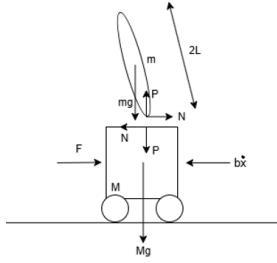


Fig. 2. Free-body diagram of the model

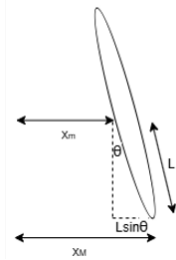


Fig. 3. The difference between two distances

The following detail is important for two variables used in the equations of motion. When a force is exerted on the cart, the distance travelled by the cart and the distance travelled by the pendulum differ with respect to their centers of mass [8]. We call the distance traversed by the cart  $x_M$  and the distance traversed by the pendulum  $x_m$ . The  $x_m$  distance lags behind  $x_M$  by the right-angled triangle component shown in Fig. 3.

There are several conclusions about the equations of motion. Equation 1 states that the force must accelerate the total mass, overcome friction and compensate for the pendulum's angular acceleration. Equation 2 establishes a relationship between the two accelerations, where the cart must accelerate to produce the desired angular acceleration. The gravity term affects the tilt of the angle. The equations have been simplified by using

the small angle approximations shown in [4]. Let  $\theta = \phi + \pi$  such that  $\phi$  is the deviation from the equilibrium [4].

$$(M + m)\ddot{x}_M + b\dot{x}_M - mL\ddot{\phi} = F \quad (1)$$

$$(I + mL^2)\ddot{\phi} - mgL\phi = mL\ddot{x}_M \quad (2)$$

The state-space representation of the system is shown below. It is obtained by rearranging the equations of motion into first-order differential equations, as shown in [9]. The transfer functions that represent the linearized input–output dynamics are shown in Proofs.

$$\dot{x} = Ax + Bu \quad (3)$$

$$y = Cx + Du \quad (4)$$

$$(5)$$

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & -\frac{(I + mL^2)b}{I(M + m) + MmL^2} & \frac{m^2gl^2}{I(M + m) + MmL^2} & 0 \\ 0 & 0 & 0 & 1 \\ 0 & -\frac{mLb}{I(M + m) + MmL^2} & \frac{mgl(M + m)}{I(M + m) + MmL^2} & 0 \end{bmatrix} \quad (6)$$

$$B = \begin{bmatrix} 0 \\ \frac{I + mL^2}{I(M + m) + MmL^2} \\ 0 \\ \frac{mL}{I(M + m) + MmL^2} \end{bmatrix} \quad (7)$$

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \quad (8)$$

$$D = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad (9)$$

The objective is to design and compare LQR, MPC,  $H_\infty$ , and Fuzzy controller that stabilize the pendulum about the upright equilibrium. The settling time and control effort of each controller will be assessed to evaluate the strengths and weaknesses of each alternative.

#### A. Energy Metric

To evaluate the efficiency of the different controllers, we use a discrete-time control-effort (energy) metric, shown on the left-hand side of Equation 10. [10]

$$\sum_{k=0}^{N-1} u(k)^2 \Delta t \approx \int_0^T u(t)^2 dt \quad (10)$$

Here,  $\Delta t$  is the sampling time and  $u(k)$  is the applied control input (force) at time step  $k$ . We choose this metric because it provides a clear and consistent basis for comparing controllers. It is always non-negative, accumulates over time, and directly reflects how aggressively the controller uses the actuator.

In contrast, a "force-times-distance" metric depends strongly on the specific cart motion. A controller may apply large forces while moving only a short distance, or apply small forces over a long distance, and vice versa. This makes the results harder to interpret and compare across different control strategies.

### B. Implementation in simulation

In PyBullet, the controller is evaluated at each simulation step using the full state  $\mathbf{x}(t)$ , which contains  $[x, \dot{x}, \phi, \dot{\phi}]$ . In our system, the position of the cart is limited to  $\pm 2m$ .

The system is initialized with the pole at approximately  $20^\circ$  from the upright position, and an external "kick"-disturbance is applied every 5 s for 0.1 s. This tests both recovery from an initial condition and disturbance rejection under identical conditions across controllers. The friction force  $b$  is the simulated air resistance that is only used for  $H_\infty$ . A detailed explanation is provided in the Controller Design section. The test- and plotting-code (plotter.py, test.py and some segments of pendulumSim.py) were generated by ChatGPT to reduce the time spent on redundant tasks.

## III. CONTROLLER DESIGN

### A. LQR

LQR computes the control action by solving a discrete-time linear-quadratic regulator problem once (offline) for the linear model [11]

$$x_{k+1} = Ax_k + Bu_k. \quad (11)$$

The solution is a constant state-feedback gain  $K$  [11] such that the online control law is

$$u_k = -Kx_k, \quad (12)$$

which minimizes the infinite-horizon quadratic cost [11]

$$J = \sum_{k=0}^{\infty} (x_k^\top Q x_k + u_k^\top R u_k), \quad (13)$$

with  $Q \succeq 0$  weighting state deviation and  $R \succ 0$  weighting control effort.

The cart-pole dynamics are inherently nonlinear and LQR is a linear controller for linear systems. We apply LQR to the linearized model dynamics about the "zero" state. We can do this because, for sufficiently small deviations around the equilibrium, the nonlinear system is well approximated by its linearization, so the LQR feedback can stabilize the system locally [4].

In our implementation, we form continuous-time  $A$  and  $B$  from the physical parameters and discretize them with a zero-order hold (piecewise-constant input over each sampling interval) to obtain the discrete  $A_d$  and  $B_d$  [11]. We then compute  $K$  from the steady-state solution of the discrete algebraic Riccati equation (DARE) [11] and apply  $u = -Kx$  at each step. The computed input is finally limited by  $\pm u_{\max}$ .

The weights are implemented as  $Q_{dt} = Q * dt$  and  $R_{dt} = R * dt$ . Scaling both  $Q$  and  $R$  by the same positive constant does not change the optimal feedback gain  $K$  (it primarily

rescales the value function / Riccati solution  $P$ ), but it is a convenient way to keep the discrete-time cost consistent with a continuous-time integral interpretation.

Note that LQR itself is an unconstrained optimal control law. When we saturate the optimal control output to  $\pm u_{\max}$ , the resulting saturated controller is no longer strictly the unconstrained optimal LQR solution.

### B. MPC

We have a MPC that computes the control action by solving, at each sampling instant, a finite-horizon optimal control problem initialized with the current measured state. The optimizer returns a control sequence, and only the first input is applied before the problem is solved again at the next step (receding-horizon principle). [12]

In our cart-pole implementation, we use a nonlinear MPC formulation with a quadratic tracking objective of the standard form

$$V_N(x) = \sum_{k=0}^{N-1} \mathcal{L}(x(k), u(k)) + V_f(x(N)) \quad (14)$$

which is the sum of the stage costs over the horizon  $N$  plus a terminal penalty. [12] The stage cost weighs deviations in the state  $[x, \dot{x}, \phi, \dot{\phi}]$  via  $Q$ , and penalizes control effort via  $R$ . The input is constrained by the hard constraint  $u_{\max}$ . Cart travel limits are handled as soft constraints by adding large penalty terms when the cart enters a soft-zone. [12]

The MPC optimization is implemented using a direct sequential approach. The decision variables are the control inputs, and the predicted trajectory is generated by forward simulation inside the objective function. [12] We discretize the continuous dynamics with a fixed-step RK4-integrator over the sampling time, and solve the resulting nonlinear program online using a bound-constrained optimizer. [12] Specifically, we use L-BFGS-B, a limited-memory BFGS method that enforces box constraints on each input over the horizon. [13]. To reduce computation, we have a warm-start for the solver by shifting the previously optimal input sequence forward by one step. [12] The cost uses quadratic state and input penalties  $(x_k - x_{\text{ref}})^\top Q (x_k - x_{\text{ref}}) + Ru_k^2$  with  $Q = \text{diag}(10, 0.1, 40, 1)$  and  $R = 0.01$ . To prevent the optimizer from continuously driving the cart towards the edge, we included a quadratic soft-zone penalty  $|x| > 1.5$  (weight  $10^4$ ), which encourages the MPC to change direction when needed to balance the pole. In addition, a stronger quadratic penalty is associated with the hard bounds  $|x| \leq 2$  (effective weight  $10^5$ ), making edge trajectories highly unfavorable. Our horizon is set to 40.

### C. Fuzzy-PD

We have implemented a hybrid control scheme that combines a Mamdani Fuzzy controller with a PD controller. There exists different types of Fuzzy controllers. Of the different Fuzzy-controllers, the Mamdani Fuzzy controller is the most common. [14] The basic idea behind Fuzzy logic is that "true" and "false" aren't just 0 or 1 like in normal boolean logic. In Fuzzy logic, the values can be anything between 0 and 1,

which represents how much something belongs to a set (its degree of membership). [15] [16] This allows the controller to have a nice property, which is to represent uncertainty and gradual transitions more naturally than Boolean logic. [17]

A Mamdani Fuzzy controller creates a nonlinear relationship between inputs and outputs using a set of if-then rules. [14] In our project, the inputs are the pendulum angle and angular velocity, and the output is the force. A nice feature of this approach is that it doesn't require a super detailed model of the system. Most of the "control strategy" comes from the membership functions and the rule base. [18]

According to [17], membership functions are curves that define how each point in the input space is mapped to a membership value (a degree of membership between 0 and 1). The input space is called "the universe of discourse" [17]. Consider the example where people are classified as "tall" or "not-tall". Rather than using a strict, or crisp, function that labels people as "tall" only if their height is at least two meters and "not-tall" otherwise, fuzzy logic assigns degrees of tallness. The crisp function is a step function where  $y = 0$  for all values  $x < 2$ . In contrast, the membership function of a fuzzy set is more nuanced: it can assign intermediate values such as 0.4 for "somewhat tall" and 0.8 for "pretty tall". Membership functions have several shapes, including piecewise-linear functions such as triangular and trapezoidal functions, and smooth functions such as Gaussian or sigmoid functions [17]. Our implementation uses the first variant, since they are simpler to interpret.

Here are illustrations of the membership functions:

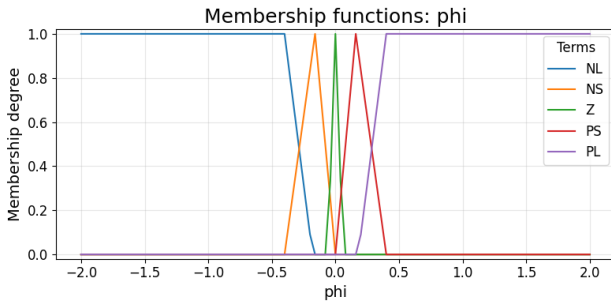


Fig. 4. Membership functions for the angle

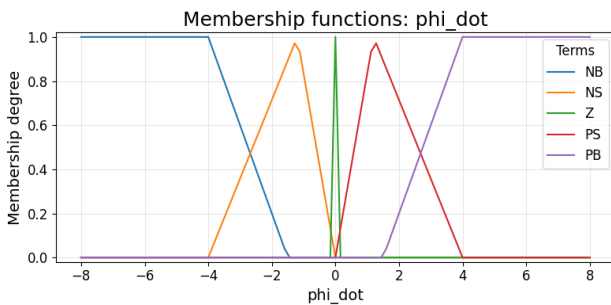


Fig. 5. Membership functions for the angular-velocity

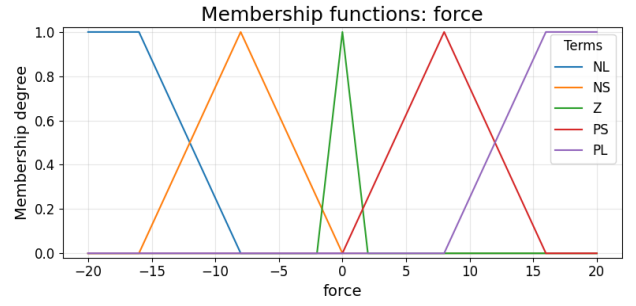


Fig. 6. Membership functions for the force

The controller has four main steps: fuzzyfication, rule evaluation, aggregation, and defuzzification. [14] [19] First, in the fuzzification stage, the numerical inputs are mapped through one or more membership functions associated with the fuzzy sets Negative Small/Large/Big, Zero and Positive Small/Large/Big. This is to obtain the degrees of membership. These degrees are the fuzzy values used by the rule base. Let's consider our implementation as an example. As stated before,  $\phi$  is the deviation from the equilibrium. Let  $\phi = 0.10$  and  $\dot{\phi} = 0.50$ . The angle  $\phi$  has membership in the PS triangular function  $[0, 0.16, 0.4]$ , and  $\dot{\phi}$  has membership in the PS triangular function  $[0, 1.2, 4]$ . Thus,  $\phi$  is fuzzified as  $\frac{0.10-0}{0.16-0}$  and  $\dot{\phi}$  as  $\frac{0.50-0}{1.2-0}$ , yielding the membership degrees 0.625 and 0.417. This example activates a single rule, but depending on the angle and angular velocity, multiple sets may be active simultaneously [19].

Then, in the rule evaluation stage, we apply the if-then rules. We combine the fuzzified antecedent conditions, using fuzzy logical operators, to compute how strongly each rule should activate (the firing strength) [19]. In our controller, we primarily use AND between our pole-angle,  $\phi$ , and the angular velocity,  $\dot{\phi}$ . By printing the rule base, we observe that the AND-operator uses the aggregation function `fmin` from the `numpy`-library, which takes the element-wise minimum of array elements and returns the array consisting of these minimums [20]. This corresponds to taking the minimum of the antecedent membership degrees. In other words, if the two antecedent membership degrees are  $\mu_1$  and  $\mu_2$ , the firing strength becomes  $\alpha = \min(\mu_1, \mu_2)$ .

The firing strength is used in the implication step, where it is applied to the consequent output fuzzy set for the force [19]. With the Mamdani-minimum functionality, it truncates the output fuzzy sets [19]. Here is an illustration from [19] that shows the rule evaluation with the OR-operator.

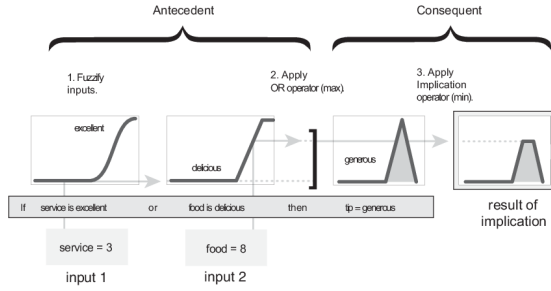


Fig. 7. Rule evaluation with OR

In the aggregation stage, we combine the results from all active rules into one Fuzzy output. According to [21], the aggregation method used in the Scikit-Fuzzy library is the max. An illustration of the use of the max-aggregation-operation is illustrated in Fig. 8 [19].

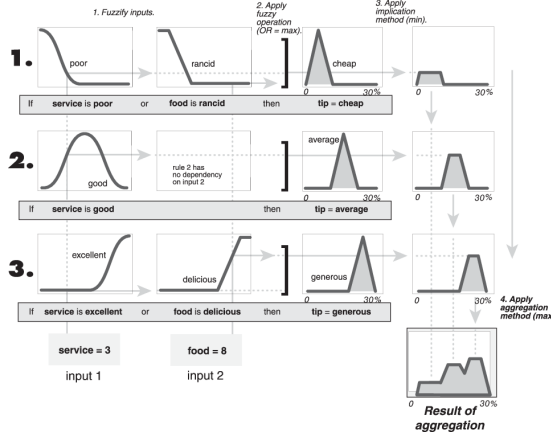


Fig. 8. Rule evaluation with OR

Finally, in the defuzzification stage, we convert the Fuzzy output into a single real force value. [14] There exists several defuzzification methods, but the one we implemented is the centroid-method. It returns the center of the area under the aggregate fuzzy set [19]. This happens through the formula

$$x_{\text{Centroid}} = \frac{\sum_i \mu(x_i) x_i}{\sum_i \mu(x_i)} \quad (15)$$

where the  $\mu(x_i)$  is the membership value for point  $x_i$  in the universe of discourse. [22]

Our Mamdani Fuzzy controller uses the pendulum angle error and angular velocity to compute the output force to balance/stabilize the pole.

The PD controller regulates the cart position by generating a small lean-angle  $\phi_{\text{ref}}$  based on the cart position and velocity. If the cart is off-center or moving,  $\phi_{\text{ref}}$  is chosen to drive the cart back towards equilibrium. Then this reference-angle is converted into the Fuzzy controller's target via  $\phi_{\text{err}}$ .

A soft wall is implemented to add a repelling force when the cart approaches the track limits.

After some trial and error, we ended up with the gain values and rule set shown in the code.

#### D. $H_\infty$

This robust control method is used to generate an optimized controller with respect to the  $H_\infty$  norm [23]. The  $H_\infty$  norm is a performance metric, defined as the worst-case gain of the closed-loop system [23]. In the implementation, we used the mixed-synthesis method `mixsyn` to impose additional constraints that tune tracking, the input sensitivity and the noise attenuation [24] [25]. There are 3 tunable weights that act as frequency domain filters [26]. These weights shape the sensitivity function  $S$ , the control effort  $KS$  and the complementary sensitivity function  $T$ .  $S$  must be small over the target frequency range [26].  $T$  must be close to 0 db at low frequencies and roll off quickly at higher frequencies to avoid amplifying noise [27].  $KS$  should also have a high-frequency roll-off to prevent control signals from having intractable magnitudes [27].

$$\|S\|_\infty \leq \gamma \|W_p^{-1}\|_\infty \quad (16)$$

$$\|KS\|_\infty \leq \gamma \|W_u^{-1}\|_\infty \quad (17)$$

$$\|T\|_\infty \leq \gamma \|W_t^{-1}\|_\infty \quad (18)$$

$$\left\| \begin{bmatrix} W_p S \\ W_u KS \\ W_t T \end{bmatrix} \right\|_\infty \leq \gamma. \quad (19)$$

$\gamma$  is the  $H_\infty$  norm of the closed-loop system and the upper bound of the weighted sum [24]. Tuning the weights manually was a cumbersome process. Although the ratio between each function and the inverse of their respective weights could estimate to 1 and stay below  $\gamma$  for certain instances, this didn't ensure that the gains were minimized. Additionally, trying to minimize the sensitivity gain usually resulted the complimentary sensitivity to deviate, and vice versa. In the resulting implementation, the maximum  $S$  norm (also known as maximum sensitivity) leads to oscillations [23]. The performance of the implemented system will vary with respect to the maximum control effort allowed and the magnitude of the disturbance.

Furthermore, we made a minor adjustment to the system dynamics due to a challenging aspect of the  $A$  matrix. One of the eigenvalues of  $A$  is 0, consequently, a pole is located at the origin. `mixsyn` requires that the following matrix should be right-invertible for  $\omega$  [28]:

$$\begin{bmatrix} A - j\omega I & B1 \\ C2 & D21 \end{bmatrix} \quad (20)$$

As a result, the  $A(0,0)$  element is non-zero for this implementation, moving the pole away from the origin. Although this method bypasses the rank deficiency issue, it changes the pole location. The friction is non-zero due to the same rank-sufficiency condition as well.

#### IV. RESULT AND DISCUSSION

We simulated every controller for 10 s and iterated between four different values for  $u_{\max}$ . This is repeated for two kick-magnitudes: 5 N and 15 N. In Fig. 7 and 8 we illustrate the variation in pendulum angle per controller. The plots show from top to bottom the open-loop response (no controller), LQR, MPC, Fuzzy-PD and  $H_{\infty}$ . The blue region in the middle shows where the kick is applied.

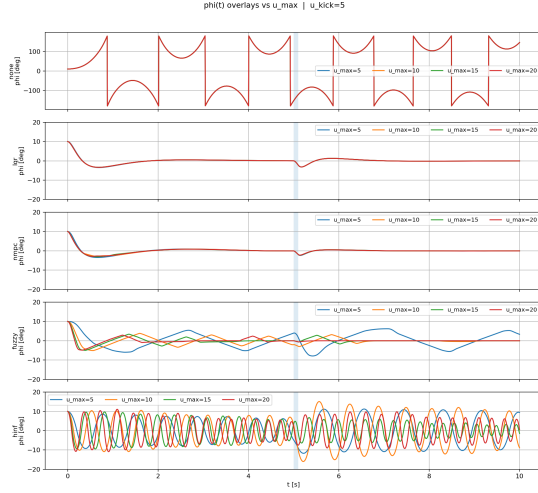


Fig. 9. Pendulum Angle when  $u_{\text{kick}}=5$

Fig. 9, where  $u_{\text{kick}}$  is 5 N, shows that the response of LQR and MPC are almost identical. They both recover quickly after the disturbance is applied. Fuzzy-PD has a more varied stabilizing performance for the different  $u_{\max}$  values. As the  $u_{\max}$  values increase, the Fuzzy-PD becomes more stable. We can also see that it stabilizes itself quicker with higher power.  $H_{\infty}$  oscillates in a  $20^{\circ}$  range and gradually dampens before the disturbance. Post-disturbance, the oscillations show irregular amplitudes with  $u_{\max}=10$  deviating most from the target angle.  $u_{\max}=15$  dampens quickest.

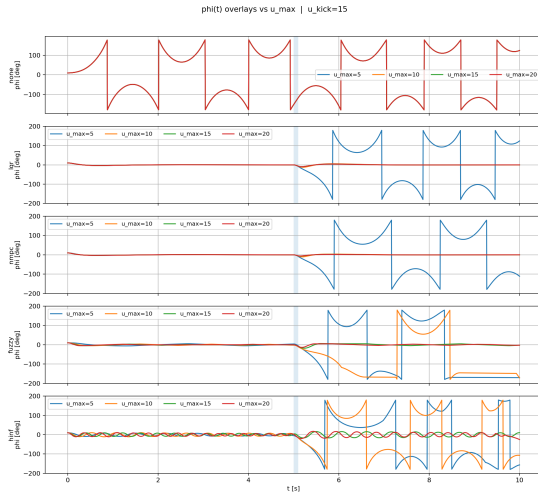


Fig. 10. Pendulum Angle when  $u_{\text{kick}}=15$

In Fig. 10, the sharp transitions correspond to the pendulum's transition between  $\pm\pi$ . This happens when the pendulum swings below the cart. LQR and MPC fail to keep the pendulum upright when  $u_{\max}=5$ , but stabilize quickly when  $u_{\max}$  is increased. The Fuzzy-PD controller responds well when  $u_{\max}$  is at least 15.  $H_{\infty}$  responds poorly when  $u_{\max}$  is 5 or 10, letting the pendulum swing between  $\pm\pi$  continuously. However, when  $u_{\max}$  is at least 15, it sustains damped oscillations.

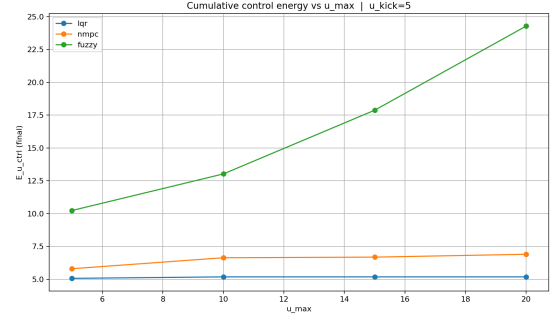


Fig. 11. Control Effort when  $u_{\text{kick}}=5$

Although the change in angular displacement look similar between LQR and MPC in Figures 9 and 10, LQR uses a smaller control effort in Figures 11 and 12 even as  $u_{\max}$  increases. This difference may be linked to how LQR and MPC operate. LQR solves the optimization task once and produces a fixed gain, whereas MPC re-solves the optimization problem at every timestep and shifts its window. The control effort of the Fuzzy controller is largest overall, with the small exception for the case when  $u_{\max}=5$  and  $u_{\text{kick}}=15$ . As we observed for Figure 9, the Fuzzy controller stabilizes quicker when  $u_{\max}$  is high. The Fuzzy controller tends to use higher control effort than LQR or MPC because it is rule-based – it doesn't explicitly optimize a cost function. Its output is generated directly from membership functions and if-then rules, so the commanded output may become large when the rules favor high values. As a consequence, this leads to higher overall energy usage.

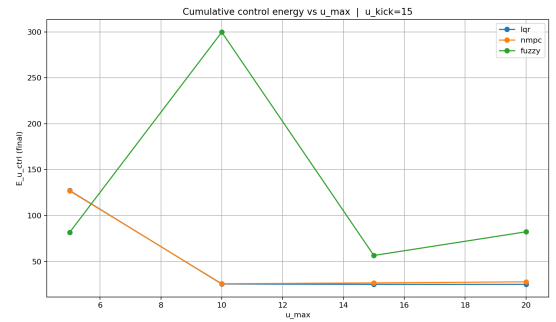


Fig. 12. Control Effort when  $u_{\text{kick}}=15$

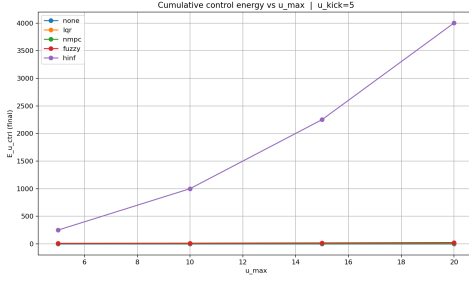


Fig. 13.  $H_\infty$  control Effort when  $u_{\text{kick}}=5$

In Fig. 13, we see that  $H_\infty$  diverges significantly from the other controllers. As  $u_{\text{max}}$  is increased, the control output increase as well, alternating between two extremes. This is shown in Fig. 14 and 15. This behavior can be attributed to a poorly conditioned  $KS$  norm, making the control output prone to noise and infeasible in magnitude.

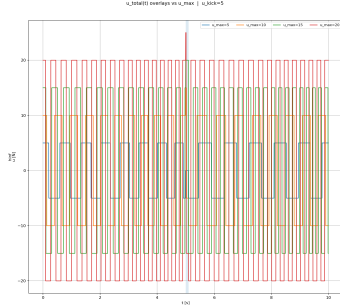


Fig. 14. Total force used when  $u_{\text{kick}}=5$  (includes kick)

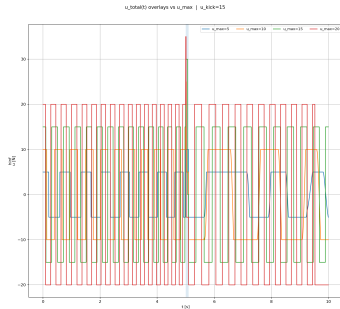


Fig. 15. Total force used when  $u_{\text{kick}}=15$  (includes kick)

The settling time evaluates the controller's ability to stabilize itself after a disturbance is applied. Our settling criteria is as follows: the controller must hold the angle steady within a  $\pm 2^\circ$  tolerance relative to the equilibrium and sustain this for 2 seconds.  $H_\infty$  is excluded from this evaluation as it oscillates continuously outside the tolerance range for our test cases.

TABLE II  
SETTLING TIME WHEN  $u_{\text{kick}}$  IS 5 N

| $u_{\text{max}}$ (N) | LQR  | MPC  | Fuzzy-PD |
|----------------------|------|------|----------|
| 5                    | 2.31 | 2.17 | X        |
| 10                   | 2.31 | 2.20 | 3.01     |
| 15                   | 2.31 | 2.20 | 2.61     |
| 20                   | 2.31 | 2.16 | 2.11     |

TABLE III  
SETTLING TIME WHEN  $u_{\text{kick}}$  IS 15 N

| $u_{\text{max}}$ (N) | LQR  | MPC  |
|----------------------|------|------|
| 10                   | 3.58 | 3.55 |
| 15                   | 3.41 | 3.02 |
| 20                   | 3.41 | 3.02 |

The Fuzzy controller converges quickly when  $u_{\text{max}}$  is increased in Table II. However, it doesn't settle in the simulation duration if  $u_{\text{max}}$  is too low or  $u_{\text{kick}}$  is too high. Hence, its exclusion from Table III. We ran an extended simulation to assess when the Fuzzy controller settles for the second test case. As seen in Fig. 17, when  $u_{\text{max}}$  is 20, the controller settles in approximately 9 seconds. This may be attributed to how the rules are defined in the implementation. Another thing to note is that our implementation of the controller responds better to kicks in the positive direction. Overall, these results indicate that the Fuzzy controller is highly sensitive to disturbance direction, and that improving robustness for low  $u_{\text{max}}$  and large kicks likely requires re-tuning the membership functions and the rule base.

As mentioned earlier, MPC optimizes over a finite prediction horizon, repeatedly recomputing the control sequence to account for future behavior and constraints. LQR, on the other hand, solves the optimization problem once for a linearized model and applies the resulting static feedback law. Therefore, it does not explicitly plan ahead or re-optimize as the state evolves across different regions of the state space. Thus, we attribute the improvement in settling time of the MPC to the aforementioned properties.

In addition to the one time optimization, LQR does not account for  $u_{\text{max}}$  in the cost function. Since  $u_{\text{kick}}$  is quite low, the control input never exceeds  $u_{\text{max}}$ . As a consequence, the LQR settling time stays consistent as  $u_{\text{max}}$  varies as seen in Table II.

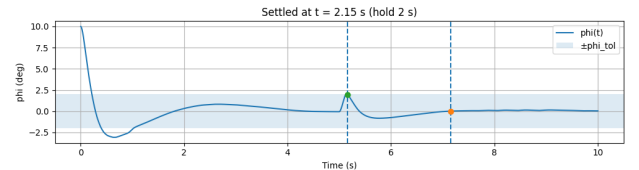


Fig. 16. MPC behavior when  $u_{\text{max}}=15$  and  $u_{\text{kick}}=5$

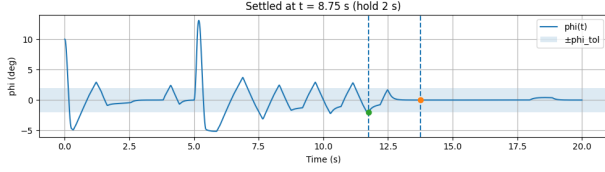


Fig. 17. Fuzzy-PD behavior when  $u_{\max}=20$  and  $u_{\text{kick}}=15$

We also simulated the MPC starting from an initial  $\phi = 170^\circ$  (almost straight down), without  $u_{\text{kick}}$ . The goal was to test whether the controller could swing the pendulum up to the equilibrium point and then stabilize it there.

To encourage the swing-up maneuver, we reduced the state-cost weight on the cart position from  $Q_x = 10$  (used earlier in our tests) to  $Q_x = 1$ . With a smaller penalty on  $x$ , the MPC is allowed to move the cart more aggressively, which helps it generate enough momentum to bring the pendulum upward.

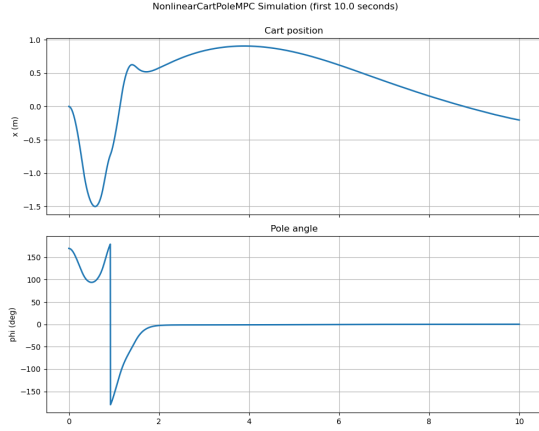


Fig. 18. Cart position  $x$  and pendulum angle  $\phi$  as the MPC swings the pendulum upright and then keeps it balanced.

As shown in Fig. 18, the controller brings the pendulum angle to the equilibrium ( $\phi \approx 0$ ) relatively quickly. However, the cart position  $x$  returns to the center ( $x \approx 0$ ) slower. This is expected, since the reduced position weight  $Q_x$  makes centering the cart a lower priority than stabilizing the angle.

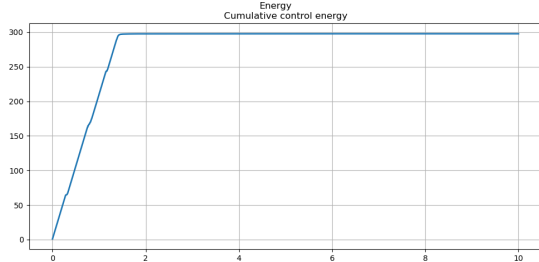


Fig. 19. Cumulative control effort (energy) over time during MPC swing-up and stabilization.

Fig. 19 highlights this trade-off: the control effort increases during the swing-up phase, when large actions are needed to

drive the pendulum upward. Once the pendulum is close to upright, the MPC reduces the control effort significantly.

This swing-up task is a *nonlinear, large-angle* maneuver that requires planning. Among the controllers we tested, MPC is the only one capable of performing this behavior without an external kick. Since it optimizes a sequence of control actions over a prediction horizon it can build up momentum to swing-up. In contrast, the other controllers in our comparison are primarily designed for stabilization and robustness in an operating region. They do not inherently plan over a horizon. As a result, they are not able to reliably swing the pendulum up.

## V. PROOFS

The transfer functions are written in the Laplace domain, meaning for each derivative the appropriate expression is multiplied by  $s$ . They show the relationship between the applied force  $U(s)$  (which was  $F(t)$  in our notation), the pendulum angle  $\Phi(s)$  and the cart position  $X(s)$  (which was  $x_M$  before).

$$\frac{\Phi(s)}{U(s)} = \frac{\frac{ml}{q}s}{s^3 + \frac{b(I+ml^2)}{q}s^2 - \frac{(M+m)mgl}{q}s - \frac{bmgl}{q}} \quad (21)$$

$$\frac{X(s)}{U(s)} = \frac{\frac{(I+ml^2)s^2 - gml}{q}}{s^4 + \frac{b(I+ml^2)}{q}s^3 - \frac{(M+m)mgl}{q}s^2 - \frac{bmgl}{q}s} \quad (22)$$

## VI. CONCLUSION AND FUTURE WORK

This project explores four different controller designs for balancing an inverted pendulum on a cart. LQR and MPC preserved fast convergence, disturbance rejection and efficient control effort. The Mamdani Fuzzy Controller showed better recovery with higher  $u_{\max}$  values. Consequently, it required a higher control effort than LQR and MPC. Although,  $H_\infty$  managed to balance the pendulum about the equilibrium, it sustained a constant oscillation. As a result, the control effort grew exponentially with increasing  $u_{\max}$ . Overall, controllers with an optimization objective outperformed alternative controllers.

In the future, a revised implementation of  $H_\infty$  and the Mamdani Fuzzy Controller will be inspected. A gradient descent-type algorithm may be developed to find the optimal weight combinations for the  $H_\infty$ -norms. Methods such as Cross-Entropy may be used to optimize the Fuzzy-PD controller as shown in [7]. Finally, repetitive trials can be applied for all four controllers to improve assessment quality.

## REFERENCES

- [1] O. J. Elle and R. Wood, *Introduction to Robotics (IN3140/IN4140)*, lecture notes, Dept. of Informatics, University of Oslo, Oslo, Norway, Spring 2025.
- [2] Sunday AI, “Machine learning and AI learning platform.” [Online]. Available: <https://www.sunday.ai>. Accessed: Dec. 17, 2025.
- [3] M. Mirkovic, “Arduino two-wheel self-balancing robot,” Arduino Project Hub. [Online]. Available: <https://projecthub.arduino.cc/mircemk/arduino-two-wheel-self-balancing-robot-edef61>. Accessed: Dec. 17, 2025.

- [4] Control Tutorials for MATLAB and Simulink (CTMS), "Inverted Pendulum: System Modeling." [Online]. Available: <https://ctms.engin.umich.edu/CTMS/index.php?example=InvertedPendulum&section=SystemModeling>. Accessed: Nov. 20, 2025.
- [5] M. Stedt, I. Zhavzharov, and H. S. Abbas, "Parameter identification and LQR/MPC balancing control of a ballbot," in *Proc. IEEE Conf. on Control Applications*, 2021. Available: <https://doi.org/10.23919/ECC55457.2022.9837996> Accessed: Dec. 17 2025
- [6] M. R. Fikri, D. W. Djamari, and S. M. Levy, "H-infinity controller design of two-wheeled mobile robot under disturbance," in *Proc. Int. Conf. on Mechanical Engineering*, 2021. Available: <https://doi.org/10.1109/ISITIA52817.2021.9502218>. Accessed: Dec. 16 2025
- [7] D. N. Anisimov, T. S. Dang, S. Banerjee, and T. A. Mai, "Design and implementation of Fuzzy-PD controller based on relation models: A cross-entropy optimization approach," *The European Physical Journal Special Topics*, vol. 226, pp. 2393–2406, Apr. 2017, doi: 10.1140/epjst/e2017-70069-y. [Online]. Available: <https://doi.org/10.1140/epjst/e2017-70069-y>. Accessed: Dec. 17, 2025.
- [8] F. Khoshnoud, "Inverted pendulum: Equations of motion," California State Polytechnic University, Pomona, CA, USA, Mechatronics lecture notes, PDF. [Online]. Available: [https://www.cpp.edu/faculty/fkhoshnoud/mechatronics\\_lecture-4-1-inverted-pendulum\\_equations\\_of\\_motion.pdf](https://www.cpp.edu/faculty/fkhoshnoud/mechatronics_lecture-4-1-inverted-pendulum_equations_of_motion.pdf). Accessed: Nov. 20, 2025.
- [9] X. Chen, "Inverted pendulum modeling," lecture handout, Dept. of Mechanical Engineering, University of Washington, Seattle, WA, USA, Dec. 2016.
- [10] F. J. Goforth, "On motion control design and tuning techniques," in *Proc. Amer. Control Conf. (ACC)*, Boston, MA, USA, Jun. 30–Jul. 2, 2004, pp. 716–721, doi: 10.23919/ACC.2004.1383689. [Online]. Available: <https://doi.org/10.23919/ACC.2004.1383689>. Accessed: Dec. 17, 2025.
- [11] P. Dorato and A. H. Levis, "Optimal Linear Regulators: The Discrete-Time Case," *IEEE Transactions on Automatic Control*, vol. 16, no. 6, pp. 613–620, 1971. Available <https://ieeexplore.ieee.org/document/1099832>
- [12] J. B. Rawlings, D. Q. Mayne, and M. Diehl, *Model Predictive Control: Theory, Computation and Design*, 2nd ed. Santa Barbara, CA, USA: Nob Hill Publishing, 2017.
- [13] Scipy v1.16.2 Manual "minimize(method='L-BFGS-B')." [Online]. Available: [https://docs.scipy.org/doc/scipy/reference/optimize.minimize-lbfgsb.html?utm\\_source=chatgpt.com](https://docs.scipy.org/doc/scipy/reference/optimize.minimize-lbfgsb.html?utm_source=chatgpt.com). Accessed: Dec. 18, 2025.
- [14] V. B. Watkar, "Fuzzy Inference Systems: Types & Applications," *Journal of Emerging Technologies and Innovative Research (JETIR)*, vol. 9, no.12, pp. g347–g351, Dec. 2022. [Online]. Available: <https://www.jetir.org/papers/JETIR2212653.pdf>. Accessed: Dec. 16, 2025.
- [15] L. A. Zadeh, "Fuzzy sets," *Information and Control*, vol. 8, no. 3, pp. 338–353, Jun. 1965, doi: 10.1016/S0019-9958(65)90241-X. [Online]. Available: [https://doi.org/10.1016/S0019-9958\(65\)90241-X](https://doi.org/10.1016/S0019-9958(65)90241-X). Accessed: Dec. 16, 2025.
- [16] GeeksforGeeks, "Fuzzy Logic and Probability : The Confusing Terms," Jul. 04, 2021. [Online]. Available: <https://www.geeksforgeeks.org/data-science/fuzzy-logic-and-probability-the-confusing-terms/>. Accessed: Dec. 16, 2025.
- [17] MathWorks, "Foundations of Fuzzy Logic," MATLAB Help Center: Fuzzy Logic Toolbox Documentation. [Online]. Available: <https://se.mathworks.com/help/fuzzy/foundations-of-fuzzy-logic.html>. Accessed: Dec. 17, 2025.
- [18] E. H. Mamdani and S. Assilian, "An experiment in linguistic synthesis with a fuzzy logic controller," *International Journal of Man-Machine Studies*, vol. 7, no. 1, pp. 1–13, 1975, doi: 10.1016/S0020-7373(75)80002-2. [Online]. Available: <https://doi.org/10.1006/ijhc.1973.0303>. Accessed: Dec. 16, 2025.
- [19] MathWorks, "Fuzzy Inference Process," MATLAB Help Center: Fuzzy Logic Toolbox Documentation. [Online]. Available: <https://se.mathworks.com/help/fuzzy/fuzzy-inference-process.html>. Accessed: Dec. 17, 2025.
- [20] GeeksforGeeks, "numpy.fmin() in Python," GeeksforGeeks, Nov. 28, 2018. [Online]. Available: <https://www.geeksforgeeks.org/python/numpy-fmin-in-python/>. Accessed: Dec. 19, 2025.
- [21] scikit-fuzzy development team, "Source code for sk-fuzzy.control.antecedent\_consequent," scikit-fuzzy v0.3 documentation, scikit-fuzzy. [Online]. Available: [https://scikit-fuzzy.readthedocs.io/en/latest/\\_modules/skfuzzy/control/antecedent\\_consequent.html](https://scikit-fuzzy.readthedocs.io/en/latest/_modules/skfuzzy/control/antecedent_consequent.html). Accessed: Dec. 19, 2025.
- [22] The MathWorks, Inc., "Defuzzification Methods," MATLAB & Simulink Documentation. [Online]. Available: <https://se.mathworks.com/help/fuzzy/defuzzification-methods.html>. Accessed: Dec. 19, 2025.
- [23] Thierry Miquel. *Introduction to Robust Control Lecture notes -Draft*. Master. Introduction to Robust Control Lecture notes -Draft, ENAC, France. 2022, pp.148. hal-03818073
- [24] MathWorks, "mixsyn — Mixed sensitivity synthesis," MATLAB Robust Control Toolbox Documentation. [Online]. Available: <https://se.mathworks.com/help/robust/ref/dynamicsystem.mixsyn.html>. Accessed: Dec. 17, 2025.
- [25] Python Control Systems Library, "control.mixsyn documentation." [Online]. Available: <https://python-control.readthedocs.io/en/0.10.2/generated/control.mixsyn.html>. Accessed: Dec. 17, 2025.
- [26] H. Kwakernaak, "Mixed sensitivity design: An aerospace case study," *IFAC Proceedings Volumes*, vol. 35, no. 1, pp. 163–168, 2002.
- [27] K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton, NJ, USA: Princeton University Press, 2008.
- [28] MIT OpenCourseWare, "Standard LTI feedback optimization setup," 6.245 *Robust Control*, Lecture notes, Massachusetts Institute of Technology, Cambridge, MA, USA, 2011.